Métricas DORA em Repositórios Open-Source do GitHub: uma análise exploratória de 300 projetos

Arthur Soares[1] **, Guilherme Vieira**[1] **, Marcos Alberto Ferreira Pinto**[1]

[1]Curso de Engenharia de Software
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte, MG, Brasil

Abstract. *We mined the four DORA metrics from 300 popular GitHub repositories, using releases as a deploy proxy and CI failures as a failure proxy. Medians: 0.36 releases/week (Medium), lead time 146.4 h, CI failure rate 5.7%, recovery time 2.8 h. An exploratory corrective-release proxy for CFR (heuristic F1 = 0.689, below the 0.70 target) correlates strongly with deployment frequency ($\rho = 0.624$), unlike the CI proxy ($\rho = -0.017$), but it is mechanically coupled to release frequency, so we do not read it against the DORA no-trade-off thesis. Changing the operational definitions alters the overall class of 22.3–40.3% of the repositories (weighted κ 0.46–0.72).*

Resumo. *Minerarmos as quatro métricas DORA em 300 repositórios populares do GitHub, com releases como proxy de deploy e falhas de CI como proxy de falha. Medianas: 0,36 releases/semana (Medium), lead time de 146,4 h, CFR(a) de 5,7% e recuperação de 2,8 h. Um proxy exploratório de CFR por releases corretivas (heurística com F1 de 0,689, abaixo da meta de 0,70) tem correlação forte com a frequência de deploy ($\rho = 0{,}624$), ao contrário do proxy de CI ($\rho = -0{,}017$), mas é acoplado mecanicamente à frequência de releases, e por isso não o lemos contra a tese DORA de ausência de trade-off. Mudar as definições operacionais altera a categoria geral de 22,3% a 40,3% dos repositórios (κ linear de 0,46 a 0,72).*

1. Introdução

As métricas DORA (*DevOps Research and Assessment*) tornaram-se o padrão de mercado para medir o desempenho de entrega de software [Forsgren et al. 2018]. Quatro medidas descrevem duas dimensões: velocidade (*deployment frequency* e *lead time for changes*) e estabilidade (*change failure rate* e tempo de recuperação). O relatório *Accelerate State of DevOps* sustenta que as melhores equipes vão bem nas duas dimensões, isto é, que velocidade e estabilidade não são um *trade-off*.

O GitHub não registra "deploys em produção" nem "falhas em produção"; só é possível *inferir* as métricas por **proxies**: uma *release* publicada faz o papel de deploy, e uma execução de CI que falhou, o de falha. Por isso cada métrica minerada precisa de uma medida de confiança. Este trabalho **calcula as métricas DORA** de repositórios populares que usam GitHub Actions e **mede o quanto as conclusões dependem das definições operacionais**. Hipóteses informais, escritas antes de ver os dados:

H1 (RQ01). Distribuição muito assimétrica, com mediana entre uma *release* a cada duas semanas e uma por mês (*Medium*) e cauda de projetos com mais de uma por semana.

H2 (RQ02). A mediana por commit (b) será bem menor que a por *release* (a), pois um commit antigo incluído na *release* faz (a) crescer e afeta pouco (b); *lead times* medianos de semanas a poucos meses.

H3 (RQ03). O CFR (a), de CI, será maior que o (b), de *releases* corretivas, com mediana de (a) entre 10% e 30%: falhas de pipeline são comuns e raramente chegam a uma *release* corretiva.

H4 (RQ04). A maioria dos episódios se resolve em horas, com cauda longa; mediana por repositório entre 1 hora e 1 dia, e parcela relevante de episódios censurados.

H5 (RQ05). Correlação de Spearman fraca ou levemente negativa entre frequência e CFR, em linha com a tese DORA, mas com ruído por causa dos proxies.

H6 (RQ06). Linguagem e tipo de projeto terão associação mais forte que a popularidade; aplicações publicam mais que bibliotecas; idade e contribuidores terão efeitos pequenos, e a maioria das diferenças perderá significância após a correção para comparações múltiplas.

H7 (RQ07). Mais de 30% dos repositórios mudarão de categoria ao trocar a definição de deploy e as variantes de LT e CFR, com κ ponderado no máximo moderado.

2. Metodologia

2.1. Desenho, janela e amostra

Desenho. Estudo observacional e mineratório: as métricas DORA são calculadas a partir de dados públicos do GitHub (*releases*, *commits* e execuções de GitHub Actions). As RQ01 a RQ07 (e a RQ08, bônus) seguem o enunciado e correspondem às hipóteses H1 a H7 da Introdução; em RQ02 e RQ03 há duas variantes obrigatórias cada. O GitHub não registra deploys nem falhas em produção, então todas as medidas são *proxies*.

Janela e amostra. A janela de 12 meses, fixada pelo professor, vai de 2025-10-02 a 2026-10-02 (UTC, dia final inclusivo: 366 dias, 52,29 semanas). Os candidatos vêm da Search API, fatiada por estrelas a partir de 1000, sem forks nem arquivados; passam os que usam GitHub Actions e têm ao menos 5 *releases* e 50 *workflow runs* válidos na janela (Tabela 1). O enunciado não define "popular"; o piso de 1000 estrelas e a exclusão de forks e arquivados são decisão do grupo (L03), assim como a leitura de "run válido" (`event=push`, criado na janela, `conclusion` em success, failure, timed_out ou startup_failure) e de "release contada" (não rascunho, não pré-release, publicada na janela) (L04, L05). Dos 60 142 candidatos, 2677 foram avaliados em ordem alfabética de `full_name` e a avaliação parou ao atingir 300; a amostra é, portanto, o primeiro conjunto de 300 que passa nos filtros, e não é aleatória (lacuna L17).

Tabela 1. Funil de seleção (`outputs/funil.csv`).

Etapa	Entram	Saem	Restam
Candidatos avaliados	2677	—	2677
Possui Actions	2677	1219	1458
≥ 5 *releases*	1458	1038	420
≥ 50 runs válidos	420	120	300

2.2. Definições operacionais, dados ausentes e coleta

Definições operacionais. Valem as do enunciado: deploy é uma *release* publicada (`draft=false`, sem pré-release); a data do commit é `commit.author.date`; só o *default branch* e `event=push`; falha é `failure`, `timed_out` ou `startup_failure`, sucesso é `success`, e as demais conclusões são ignoradas. Deployment frequency é o número de *releases* da janela dividido por 52,29 semanas. *Lead time:* (a) publicação menos o commit mais antigo da *release*, mediana por repositório; (b) publicação menos cada commit, mediana de todos os commits do repositório, em horas, com os commits de `compare/{anterior}...{R}`. A primeira *release* da história e as com `compare` 404 são ignoradas. *Releases* sem commits novos e commits posteriores à *release* são excluídos e contados (decisão do grupo, L07). *CFR (a):* falhas divididas por falhas mais sucessos. *CFR (b):* a *release* R falha se qualquer *release* de deploy em $(R, R{+}7\text{ d}]$ for corretiva; as dos últimos 7 dias da janela ficam fora do denominador (censura). "Qualquer release no intervalo", e não só a próxima, é decisão do grupo (L08). *Recuperação:* por workflow, do início da primeira falha até o fim (`updated_at`) do próximo sucesso; mediana por repositório; episódios sem fim na janela são censurados e contados à parte. O enunciado inicia o episódio na "primeira falha após um sucesso"; o código também o inicia quando a primeira execução válida do workflow na janela é uma falha, sem sucesso anterior, e a mediana usa só os episódios concluídos (decisão do grupo, L09). *Classificação DORA:* cortes fixos da disciplina e mediana, arredondada para baixo, das quatro notas; quando uma métrica é indefinida (23 repositórios sem episódio de recuperação concluído), a geral usa as definidas e `n_metricas_dora` sinaliza o caso, e os 300 repositórios permanecem na análise (decisão do grupo, L10). *Combinações da RQ07:* C1 = release, LT(a), CFR(a); C2 = release mais pré-release, LT(b), CFR(b); C3 = tag, LT(b), CFR(a); a recuperação é a mesma. Em C2 e C3 só a unidade do DF muda de fato; LT(b) e CFR(b) seguem calculados sobre as *releases* de deploy.

Dados ausentes e censura (contados, nunca descartados em silêncio; `data/metrics.csv`): 6 *releases* ignoradas por `compare` 404 e 93 primeiras *releases* sem anterior; 381 commits (32 repositórios) posteriores à *release* e 1686 *releases* (51 repositórios) sem commits novos, fora do *lead time*; 418 *releases* dos últimos 7 dias da janela, fora do denominador do CFR (b); episódios censurados e repositórios sem recuperação na Seção 3.4.

Coleta. Script próprio, via API REST, sem bibliotecas de acesso ao GitHub; cache em disco e retomada; espera pelo *rate limit* e *backoff* exponencial para erros 5xx. Nenhum dia de execuções atingiu o teto de 1000 resultados, e o `compare` é paginado. *Releases* com `compare` divergente (backports) podem inflar o *lead time*. O cache não é versionado, e os repositórios continuam recebendo commits, então uma nova coleta pode divergir (detalhes no `README.md`).

2.3. Validação manual

Sorteamos 60 repositórios (semente 42) e 5 *releases* de cada, as mesmas para os três avaliadores. Cada um rotulou o tipo do projeto, se as *releases* são entregas reais e se cada uma é corretiva. O consenso é por unanimidade ou maioria (`PROTOCOLO_CONSENSO.md`): 85 das 300 *releases* são corretivas. O κ de Fleiss é 0,743 para o tipo do projeto (substancial), 0,352 para entrega real e 0,273 para `corretiva` (razoáveis; escala de Landis

& Koch [Landis and Koch 1977]). O κ baixo em `corretiva` indica que o próprio critério é ambíguo, o que é um achado (em entrega real, a prevalência de "sim" derruba o κ). **Limitação.** A rotulagem usou sugestões automáticas, iguais para todos os avaliadores, e os artefatos não demonstram três julgamentos independentes; o κ e o consenso podem herdar esse viés, cujo tamanho não foi medido. Um repositório da amostra (`1bananachicken/MaaNTE`) saiu do GitHub depois do sorteio, e dois avaliadores rotularam suas *releases* sem ver o conteúdo. O professor informou ao grupo que a validação vale assim, sem necessidade de nova rotulagem. Detalhes: `data/amostra_ouro/`, `docs/rotulagem-assistida.md`.

2.4. Heurística de release corretiva

A heurística é livre (enunciado, RQ03b); a v1 reproduz o exemplo do professor (muda só o *patch* e traz `fix`). A Tabela 2 traz o F1 de cada versão contra o consenso (300 *releases*). As v2–v7 combinam SemVer e palavra-chave; as v8–v9 olham commits e notas; as v10–v12 rejeitam *bump major/minor*, aceitam notas que se declaram correção e reconhecem *patch* pela tag (*backports*). As v8–v12 foram desenhadas *depois* de analisar os erros na mesma amostra em que são avaliadas, então seu F1 de desenvolvimento é otimista. Precisão e *recall* de cada versão estão em `data/amostra_ouro/avaliacao_heuristica.csv` e `docs/validacao-independente.md`.

Tabela 2. F1 de cada versão da heurística (amostra de desenvolvimento, consenso de 300 *releases*).

	v1	v2	v3	v4	v5	v6	v7	v8	v9	v10	v11	v12
F1	0,557	0,491	0,467	0,580	0,576	0,490	0,482	0,497	0,588	0,646	0,671	**0,722**

Validação independente da v12. A v12 foi congelada e avaliada uma única vez em 40 repositórios sorteados (semente 7342), disjuntos dos 60 de desenvolvimento, com 5 *releases* cada (200): precisão 0,729, *recall* 0,654 e **F1 = 0,689** (IC95% 0,563–0,790; *bootstrap* por repositório com 10 000 reamostragens); κ de Fleiss em `corretiva` 0,868. A meta de F1 $\geq$ 0,70 **não** foi atingida de forma independente, embora o intervalo a inclua. Por isso o CFR (b) é **exploratório**, uso que o professor permitiu ao grupo, e toda conclusão que o usa (RQ03b, RQ05b, C2 da RQ07 e RQ08) deve citar o F1 de 0,689 e seu intervalo.

3. Resultados e Discussão

Todas as análises partem de `data/metrics.csv` ($n = 300$ repositórios) e reportam mediana e IQR, com os cortes fixos da disciplina (Tabela 3). Release $\neq$ deploy e falha de CI $\neq$ falha em produção; o CFR (b) é **exploratório**, pois a heurística obteve F1 independente de 0,689, abaixo da meta de 0,70 (Seção 2.4). A Tabela 4 confronta as hipóteses da Introdução com os resultados.

3.1. RQ01 — Frequência de deploy

A mediana foi de **0,36 releases/semana** (Q1 = 0,19; Q3 = 0,77), pouco mais de uma *release* a cada três semanas, com cauda à direita (Figura 1). Pelos cortes da disciplina, 2,0% são *Elite*, 18,3% *High*, 47,7% *Medium* e 32,0% *Low*; a classe geral C1 é 9,3/45,0/35,7/10,0% (E/H/M/L). A H1 se confirma.

Tabela 3. Mediana, quartis, IQR e classe DORA por métrica. "Classe" classifica a mediana; E/H/M/L são os repositórios em Elite/High/Medium/Low. Recuperação: só repositórios com episódio concluído.

RQ	Métrica	n	Mediana	Q1	Q3	IQR	Classe (mediana)	E/H/M/L
RQ01	Deployment frequency (rel./sem.)	300	0,36	0,19	0,77	0,58	Medium	6/55/143/96
RQ02	Lead time (a), por release (h)	300	146,4	26,8	544,4	517,6	High	70/88/86/56
RQ02	Lead time (b), por commit (h)	300	146,3	32,5	437,0	404,6	High	66/87/95/52
RQ03	CFR (a), proxy de CI (%)	300	5,7	2,4	14,1	11,7	Elite	229/40/17/14
RQ03	CFR (b), proxy de entrega (%)	299	20,0	4,1	46,5	42,4	High	121/67/33/78
RQ04	Recuperação (h)	277	2,8	0,7	21,2	20,6	High	92/126/43/16

Tabela 4. Hipóteses $\times$ resultados (expectativas na Introdução).

H	Observado (a expectativa está na Introdução)	Confirmada?
H1	Mediana 0,36 rel./sem.; 18,3% *High* e 2,0% *Elite*	Sim
H2	Medianas iguais (146,4 h e 146,3 h); por repositório, razão LT(a)/LT(b) com mediana 1,21 (IQR 0,53–2,22)	Não
H3	CFR(a) 5,7%; CFR(b) 20,0%	Não
H4	Mediana 2,8 h; censurados: 1,9% dos episódios	Parcial
H5	CFR(a): $\rho = -0{,}017$; CFR(b): $\rho = 0{,}624$	Parcial
H6	8 de 30 testes significativos: idade (4), contribuidores (2), linguagem (2), efeito médio; tipo e popularidade, nenhum	Parcial
H7	C1$\times$C2: 40,3%, $\kappa = 0{,}46$; C1$\times$C3: 22,3%, $\kappa = 0{,}72$	Parcial

3.2. RQ02 — Lead time for changes

As medianas entre repositórios são **146,4 h** para LT(a) ($\approx$ 6,1 dias; Q1 = 26,8; Q3 = 544,4) e **146,3 h** para LT(b) (Q1 = 32,5; Q3 = 437,0), ambas *High*. Elas coincidem, o que contraria a H2, mas só no agregado: por repositório a razão LT(a)/LT(b) tem mediana 1,21 e IQR 0,53–2,22; LT(a) supera o dobro de LT(b) em 29% dos repositórios, LT(b) supera o dobro de LT(a) em 24% e só 47% ficam entre metade e o dobro (Wilcoxon pareado, $p < 0{,}001$; Figura 1). As variantes medem coisas diferentes: LT(a) usa o commit *mais antigo* de cada *release*, e LT(b) pesa mais as *releases* com muitos commits. Um commit antigo "esquecido" eleva LT(a) sem mover muito LT(b), mas 40% dos repositórios têm LT(a) $<$ LT(b), o que ele não explica. Não testamos causas: o lead time de cada commit está só no cache, não versionado.

3.3. RQ03 — Change failure rate

O CFR (a), proxy de CI, teve mediana de **5,7%** (Q1 = 2,4%; Q3 = 14,1%); o CFR (b), proxy de entrega, de **20,0%** (Q1 = 4,1%; Q3 = 46,5%; $n = 299$, pois um repositório tem menos de duas *releases* na janela). A H3 é refutada nos dois pontos. O CFR(a) conta *execuções de CI* que falharam, e falha de pipeline (teste instável, lint, ambiente) não é falha em produção; o CFR(b) conta *releases* seguidas de uma corretiva em 7 dias, segundo uma heurística imperfeita (F1 de 0,689), e mistura correções reais com erros do proxy.

3.4. RQ04 — Tempo de recuperação

A mediana foi de **2,8 h** (Q1 = 0,7; Q3 = 21,2; $n = 277$), classe *High*. **Censura:** 176 de 9.062 episódios (1,9%) não terminaram na janela; entre os 283 repositórios com episódio, a mediana da proporção de censurados é 0% (Q3 = 4,7%) e 105 têm ao menos um censurado. Os censurados não entram na mediana, mas são contados. Dos 23 repositórios

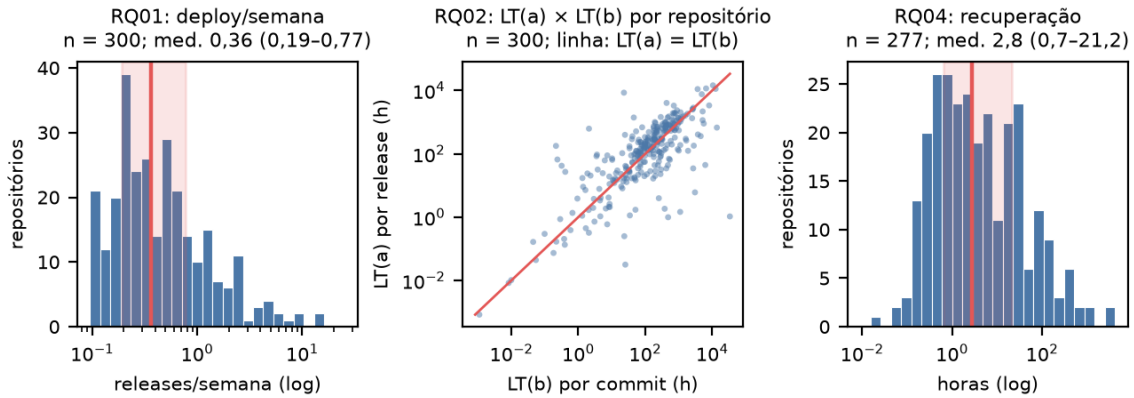


Figura 1. RQ01, RQ02 e RQ04 por repositório (escala log). Linha e faixa vermelhas: mediana e IQR.

sem valor, 17 não tiveram episódio e 6 só tiveram censurados, o que tende a subestimar a recuperação, pouco dado o volume de censura. A H4 se confirma só em parte: a mediana está entre 1 h e 1 dia, mas a censura é bem menor que a esperada.

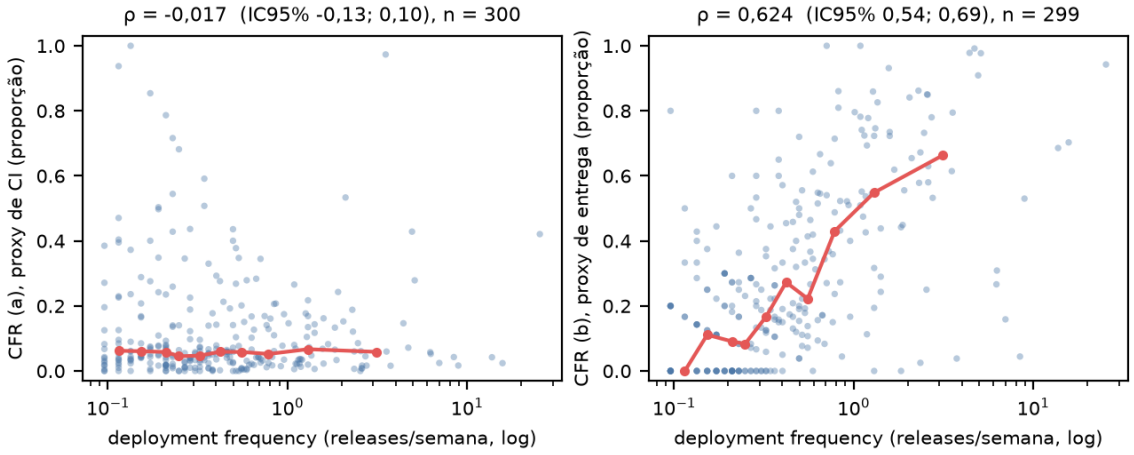


Figura 2. RQ05: DF (escala log) × CFR(a) e CFR(b); linha vermelha: mediana do CFR por faixa de DF.

3.5. RQ05 — Frequência de deploy × CFR

Com o CFR(a), $\rho = -0{,}017$ ($p = 0{,}768$; IC 95% $[-0{,}13; 0{,}10]$; $n = 300$): sem relação monotônica detectável. Com o CFR(b), $\rho = 0{,}624$ ($p < 0{,}001$; IC 95% $[0{,}54; 0{,}69]$; $n = 299$): correlação positiva forte (Figura 2); os intervalos usam a transformação z de Fisher com erro-padrão de Bonett e Wright [Bonett and Wright 2000]. A H5 se confirma só para (a). Isso não refuta a tese DORA, e correlação não implica causalidade: o CFR(b) é acoplado mecanicamente à frequência (mais *releases*, mais chances de uma corretiva em 7 dias), e com poucas *releases* há muitos zeros. Com ao menos 20 *releases* avaliadas, ρ cai para 0,42 ($n = 140$), mas isso também restringe a faixa de DF. A idade confunde (RQ06). O resultado (b) é um possível artefato, e o (a) é compatível com a tese.

3.6. RQ06 — Características dos repositórios

Testamos cinco fatores (linguagem, popularidade, contribuidores, idade, tipo; $n = 51$ no tipo) contra as seis métricas, 30 testes. Todos os fatores têm três ou mais grupos, então todos usaram Kruskal-Wallis; Mann-Whitney e o δ de Cliff [Cliff 1993] (magnitude de Romano et al. [Romano et al. 2006]) não foram exercitados com dados reais, e o δ aparece só como contraste descritivo em `rq06_testes.csv`. Os p-valores foram corrigidos por Holm [Holm 1979] sobre os 30 testes, e o efeito é $\varepsilon^2 = H/(n-1)$, com a escala de Mangiafico [Mangiafico 2016] (pequeno de 0,01 a $< 0{,}08$, médio até $< 0{,}26$; orientação do autor, não padrão; "desprezível" ($< 0{,}01$) é rótulo nosso). **8 associações** permaneceram significativas, todas de efeito médio (Tabela 5, Figura 3). Os mais jovens (Q1 de idade) publicam mais (DF 0,65 contra 0,27 rel./sem. no Q4), têm LT(a) menor (24 h contra 291 h) e CFR(b) maior (47,5% contra 12,2%), em linha com o acoplamento da RQ05; mais contribuidores, LT(a) maior (61 h no Q1 contra 291 h no Q4); Rust tem os menores LT(a) e JavaScript a recuperação mais rápida. Popularidade (menor $p_{\text{Holm}} =$ 0,60) e tipo não foram significativos, e no tipo o n limita o poder. A H6 é parcial: linguagem pesa mais que popularidade, mas o tipo não se destaca, e idade e contribuidores têm efeito médio, não pequeno. São associações, não causas.

Tabela 5. RQ06: associações significativas após Holm (Kruskal-Wallis, $\varepsilon^2 = H/(n-1)$); os 30 testes estão em `outputs/analise/rq06_testes.csv`.

Fator	Métrica	n	H	p_{Holm}	ε^2	Magnitude
Idade	Lead time (a) por release	300	64,8	$< 0{,}001$	0,217	médio
Idade	Lead time (b) por commit	300	51,2	$< 0{,}001$	0,171	médio
Idade	CFR (b) proxy de entrega	299	40,3	$< 0{,}001$	0,135	médio
Idade	Deployment frequency	300	36,2	$< 0{,}001$	0,121	médio
Contribuidores	Lead time (a) por release	300	33,8	$< 0{,}001$	0,113	médio
Contribuidores	Lead time (b) por commit	300	26,0	$< 0{,}001$	0,087	médio
Linguagem	Lead time (a) por release	298	24,9	0,0087	0,084	médio
Linguagem	Tempo de recuperação	276	22,2	0,025	0,081	médio

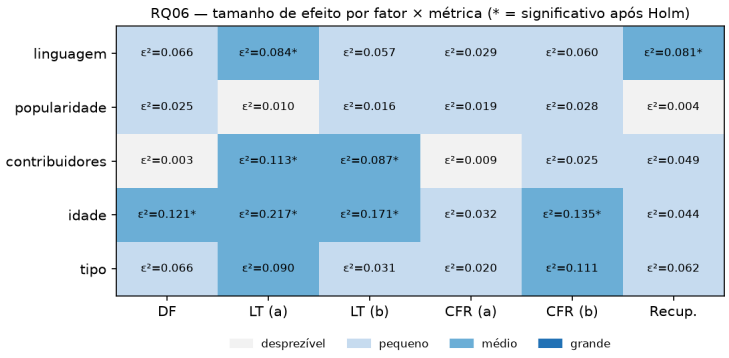


Figura 3. RQ06: ε^2 por fator e métrica (* = significativo após Holm).

3.7. RQ07 — Sensibilidade das classificações

Comparamos C1 (release, LT(a), CFR(a)), C2 (release e pré-release, LT(b), CFR(b)) e C3 (tag, LT(b), CFR(a)), com categorias codificadas de 1 a 4 e κ linear ponderado [Cohen 1968] (Figura 4). C1×C2 muda em 40,3% dos repositórios ($\kappa = 0{,}46$,

moderado), o que confirma a H7; C1×C3 em 22,3% ($\kappa = 0{,}72$) e C2×C3 em 29,3% ($\kappa = 0{,}61$), ambos substanciais, abaixo de 30%. **Limitações:** em C2 e C3 só a unidade do DF muda (LT(b) e CFR(b) seguem sobre releases), e 23, 24 e 23 repositórios em C1, C2 e C3 são classificados com três métricas.

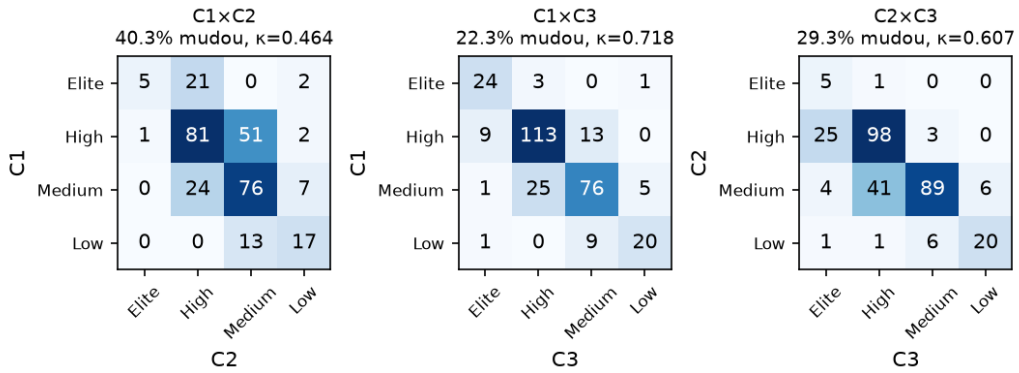


Figura 4. RQ07: matrizes de transição entre combinações (linha = primeira; coluna = segunda).

Robustez de RQ01–RQ06. A maior discordância é C1×C2, que troca CFR(a) por CFR(b) além da unidade de deploy, coerente com as RQ03 e RQ05: as conclusões sobre CFR dependem da definição, e a de DF×CFR(b) é a menos robusta. As associações de LT da RQ06 aparecem em LT(a) e LT(b). A RQ01 depende da unidade de deploy (em C1×C3, 22,3% mudam de classe), e a RQ04 não tem variante. Na análise ampliada, só quatro das sete variações reclassificam de fato (janela do CFR(b) e cortes $\pm 10\%$): mudam 2,0% a 6,0% dos repositórios, com $\kappa \geq 0{,}93$; as outras três dão 0% por construção e não são evidência de robustez. A classificação é estável a pequenas perturbações dos cortes, mas depende da unidade de deploy e da variante de CFR.

3.8. RQ08 — Rework rate (bônus)

A *rework rate* (proporção de *releases* corretivas) teve mediana de **28,6%** (Q1 = 13,5%; Q3 = 46,3%; $n = 297$), contra 20,0% do CFR(b) no subconjunto pareado. Três repositórios ficam fora (dois sem rótulos no cache, um com contadores divergentes do `metrics.csv`). A diferença pareada tem mediana 0, mas IQR de 31,4 p.p. e Wilcoxon $p = 0{,}027$; $\rho = 0{,}48$ entre as duas. As medidas não são equivalentes: o CFR(b) conta a *release* que *precisou* de correção, e o rework a que *é* correção. Ambas herdam a imprecisão da heurística v12.

4. Ameaças à validade

Seguimos as quatro categorias clássicas de Wohlin et al. [Wohlin et al. 2012].

Construto. *Release ≠ deploy*: bibliotecas publicam versões, e quem faz deploy são os usuários; projetos de entrega contínua sem *releases* ficam de fora. *Falha de CI ≠ falha em produção*: o CFR(a) (mediana 5,7%) conta testes instáveis e falhas de ambiente, e pode subestimar ou superestimar falhas reais. *Heurística corretiva*: F1 independente de 0,689 (IC95% 0,563–0,790), abaixo da meta, e κ de Fleiss de 0,273 em `corretiva`, o que mostra um critério ambíguo; todo resultado com o CFR(b) é exploratório. O CFR(b) ainda é acoplado mecanicamente à frequência de *releases* (RQ05).

Datas: `author.date` muda com *rebase* e *squash*, o que distorce o lead time; as *releases* não são filtradas pelo *default branch*, e o `compare` divergente (backports; 1371 *releases*, 102 repositórios) pode inflar o LT(a).

Interna. A v12 foi escolhida entre doze versões na mesma amostra de desenvolvimento (F1 de 0,722, otimista; o independente, 0,689, é a estimativa mais confiável). A rotulagem usou sugestões automáticas, o que reduz a independência dos avaliadores e pode inflar o consenso. Os dados ausentes não são aleatórios: os 23 repositórios sem recuperação têm poucas falhas de CI ou só episódios censurados (provavelmente os mais estáveis), e o tempo de recuperação mediano tende a ser subestimado. A idade é um fator não controlado: os mais jovens publicam mais e têm CFR(b) maior (RQ06).

Externa. Os 300 repositórios são os primeiros que passam nos filtros numa ordem alfabética global por `full_name` (`pipeline/selection.py`), entre os de 1000 estrelas ou mais: há viés de seleção, e não há projetos pouco populares nem corporativos. Predominam TypeScript (22%), Python (18%), Go (14%) e Rust (10%), e o tipo do projeto só existe para a amostra-ouro ($n = 51$ usados). A janela de 12 meses (2025-10-02 a 2026-10-02) captura um período específico.

Conclusão. A RQ06 faz 30 testes (5 fatores $\times$ 6 métricas), com Holm e $\alpha = 0{,}05$; só 8 são significativos, todos de efeito médio (ε^2 de 0,08 a 0,22), grupos com menos de 5 repositórios saem do teste, e o fator tipo tem poder baixo. O ρ de Spearman mede associação monotônica, não causal. As classificações mudam em 22,3% a 40,3% dos repositórios conforme a combinação de definições (RQ07), então comparações com estudos que usem outras definições pedem cautela.

5. Conclusão

Este trabalho calculou as quatro métricas DORA para 300 repositórios open-source do GitHub e respondeu sete questões de pesquisa. Os principais achados são: (i) a maioria dos projetos populares publica menos de uma *release* por semana (mediana 0,36; apenas 2,0% no nível *Elite*); (ii) as medianas de *lead time* por release e por commit coincidem entre repositórios, mas divergem repositório a repositório (razão mediana 1,21); (iii) o CFR por CI é baixo (5,7%) e não se associa à frequência de deploy, enquanto o CFR por heurística corretiva (20,0%, exploratório) se associa positivamente a ela, o que é compatível com um acoplamento mecânico do proxy e não permite conclusão causal; (iv) idade e contribuidores se associam a LT e DF mais que popularidade e tipo do projeto; (v) as classificações DORA são sensíveis à unidade de deploy e à variante de CFR, o que limita comparações entre estudos com definições operacionais distintas.

Trabalhos futuros: validar a heurística com amostra maior e avaliadores independentes, usar *issues* e *pull requests* como proxies e replicar o estudo em projetos corporativos.

Repositório e GitHub Projects: `https://github.com/Laboratorio-Medicao/Lab03` e `https://github.com/orgs/Laboratorio-Medicao/projects/1`.

Referências

Bonett, D. G. and Wright, T. A. (2000). Sample size requirements for estimating Pearson, Kendall and Spearman correlations. *Psychometrika*, 65(1):23–28.

Cliff, N. (1993). Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509.

Cohen, J. (1968). Weighted kappa: Nominal scale agreement provision for scaled disagreement or partial credit. *Psychological Bulletin*, 70(4):213–220.

Forsgren, N., Humble, J., and Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70. JSTOR 4615733.

Landis, J. R. and Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Mangiafico, S. S. (2016). Summary and analysis of extension program evaluation in R. Versão 1.25.0, revisada em 2026; pacote `rcompanion`.

Romano, J., Kromrey, J. D., Coraggio, J., and Skowronek, J. (2006). Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? In *Annual Meeting of the Florida Association of Institutional Research*, Cocoa Beach, FL.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg.